

Advanced SQL

Name: Nomit Bagaria

Assignment Name: Advanced SQL

Mob.No: 9828255290

Email: bagarianomit@gmail.com

Q1. What is a Common Table Expression (CTE), and how does it improve SQL query readability?

Answer:

A Common Table Expression (CTE) is a temporary named result set that is defined using the WITH keyword. It exists only for the duration of a single SQL query.

CTEs improve query readability by:

- Breaking complex queries into smaller and easier-to-understand parts.
- Improving code organization and maintenance.
- Eliminating the need to write the same subquery multiple times.
- Making recursive queries easier to write.

Q2. Why are some views updatable while others are read-only? Explain with an example.

Answer:

A view is updatable if it is based on a single table and does not contain operations such as JOIN, GROUP BY, DISTINCT, aggregate functions, or calculated columns.

A view becomes read-only when it contains complex queries involving joins, grouping, or aggregate functions because SQL cannot determine how changes should be applied to the original tables.

Example:

Updatable View:

```
CREATE VIEW vw_Product AS
SELECT ProductID, ProductName, Price
FROM Products;
```

Read-only View:

```
CREATE VIEW vw_SalesSummary AS
SELECT Category, AVG(Price)
FROM Products
GROUP BY Category;
```

Q3. What advantages do stored procedures offer compared to writing raw SQL queries repeatedly?

Answer:

Stored procedures provide several advantages over writing SQL queries repeatedly:

- Reduce code duplication.
- Improve performance because execution plans are cached.
- Increase security by allowing users to execute procedures without direct access to tables.
- Simplify application development by storing business logic in the database.
- Make maintenance easier because changes are made in one place.

Q4. What is the purpose of triggers in a database? Mention one use case where a trigger is essential.

Answer:

A trigger is a database object that automatically executes when an INSERT, UPDATE, or DELETE operation occurs on a table.

Triggers are used to:

- Maintain audit logs.
- Enforce business rules.
- Automatically update related tables.
- Track changes in data.

Example Use Case:

An AFTER DELETE trigger can automatically copy deleted product records into an archive table for future reference.

Q5. Explain the need for data modelling and normalization when designing a database.

Answer:

Data modelling is the process of designing how data is organized and how different tables are related.

Normalization is the process of organizing data to reduce redundancy and improve data integrity.

Benefits include:

- Eliminates duplicate data.
- Maintains consistency.
- Reduces storage requirements.
- Makes updates easier.
- Improves overall database performance and reliability.

SQL Code to Create the Table and insert the Data (Use for Q6–Q9) :

```
CREATE TABLE Products (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(100),
    Category VARCHAR(50),
    Price DECIMAL(10,2)
);
```

```
INSERT INTO Products
VALUES
(1, 'Keyboard', 'Electronics', 1200),
(2, 'Mouse', 'Electronics', 800),
(3, 'Chair', 'Furniture', 2500),
(4, 'Desk', 'Furniture', 5500);
CREATE TABLE Sales (
```

```

SaleID INT PRIMARY KEY,
ProductID INT,
Quantity INT,
SaleDate DATE,
FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);

```

```

INSERT INTO Sales
VALUES
(1, 1, 4, '2024-01-05'),
(2, 2, 10, '2024-01-06'),
(3, 3, 2, '2024-01-10'),
(4, 4, 1, '2024-01-11');

```

Q6. Write a CTE to calculate the total revenue for each product (Revenue = Price × Quantity), and return only products where revenue > 3000.

Answer:

```

WITH ProductRevenue AS
(
    SELECT
        p.ProductID,
        p.ProductName,
        p.Price,
        s.Quantity,
        (p.Price * s.Quantity) AS Revenue
    FROM Products p
    JOIN Sales s
    ON p.ProductID = s.ProductID
)

SELECT *
FROM ProductRevenue
WHERE Revenue > 3000;

```

```

38         p.Price,
39         s.Quantity,
40         (p.Price * s.Quantity) AS Revenue
41     FROM Products p
42     JOIN Sales s
43     ON p.ProductID = s.ProductID
44 )
45
46 SELECT *
47 FROM ProductRevenue
48 WHERE Revenue > 3000;
49

```

Result Grid					
Filter Rows:					
Export: Wrap Cell Content: IA					
	ProductID	ProductName	Price	Quantity	Revenue
▶	1	Keyboard	1200.00	4	4800.00
	2	Mouse	800.00	10	8000.00
	3	Chair	2500.00	2	5000.00
	4	Desk	5500.00	1	5500.00

Q7. Create a view named vw_CategorySummary that shows: Category, TotalProducts, AveragePrice.

Answer:

```
CREATE VIEW vw_CategorySummary AS
SELECT
    Category,
    COUNT(*) AS TotalProducts,
    AVG(Price) AS AveragePrice
FROM Products
GROUP BY Category;

SELECT * FROM vw_CategorySummary;
```

```
49
50 • CREATE VIEW vw_CategorySummary AS
51 SELECT
52     Category,
53     COUNT(*) AS TotalProducts,
54     AVG(Price) AS AveragePrice
55 FROM Products
56 GROUP BY Category;
57
58 • SELECT * FROM vw_CategorySummary;
59
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Category	TotalProducts	AveragePrice
▶	Electronics	2	1000.000000
	Furniture	2	4000.000000

Q8. Create an updatable view containing ProductID, ProductName, and Price. Then update the price of ProductID = 1 using the view.

Answer:

```
CREATE VIEW vw_ProductDetails AS
SELECT
    ProductID,
    ProductName,
    Price
FROM Products;

UPDATE vw_ProductDetails
SET Price = 1500
WHERE ProductID = 1;

SELECT * FROM vw_ProductDetails;
```

```
61 SELECT
62     ProductID,
63     ProductName,
64     Price
65 FROM Products;
66
67 • UPDATE vw_ProductDetails
68 SET Price = 1500
69 WHERE ProductID = 1;
70
71 • SELECT * FROM vw_ProductDetails;
72
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	ProductID	ProductName	Price
▶	1	Keyboard	1500.00
	2	Mouse	800.00
	3	Chair	2500.00
	4	Desk	5500.00

Q9. Create a stored procedure that accepts a category name and returns all products belonging to that category.

Answer:

```
DELIMITER //
```

```
CREATE PROCEDURE GetProductsByCategory(IN p_category VARCHAR(50))
```

```

BEGIN
    SELECT *
    FROM Products
    WHERE Category = p_category;
END //

DELIMITER ;

CALL GetProductsByCategory('Electronics');

```

```

73 DELIMITER //
74
75 • CREATE PROCEDURE GetProductsByCategory(IN p_category VARCHAR(50))
76 BEGIN
77     SELECT *
78     FROM Products
79     WHERE Category = p_category;
80 END //
81
82 DELIMITER ;
83
84 • CALL GetProductsByCategory('Electronics');
85

```

ProductID	ProductName	Category	Price
1	Keyboard	Electronics	1500.00
2	Mouse	Electronics	800.00

Q10. Create an AFTER DELETE trigger on the Products table that archives deleted product rows into a new table ProductArchive. The archive should store ProductID, ProductName, Category, Price, and DeletedAt timestamp.

Answer:

Creating Archive Table

```

CREATE TABLE ProductArchive
(
    ProductID INT,
    ProductName VARCHAR(100),
    Category VARCHAR(50),
    Price DECIMAL(10,2),
    DeletedAt TIMESTAMP
);

```

Creating Trigger

```

DELIMITER //

CREATE TRIGGER trg_ProductArchive
AFTER DELETE ON Products
FOR EACH ROW
BEGIN
    INSERT INTO ProductArchive (ProductID, ProductName, Category, Price, DeletedAt)
    VALUES (OLD.ProductID, OLD.ProductName, OLD.Category, OLD.Price, NOW());
END //

DELIMITER ;

```

Testing the Trigger

```
SET FOREIGN_KEY_CHECKS = 0;  
DELETE FROM Products WHERE ProductID = 3;  
SET FOREIGN_KEY_CHECKS = 1;
```

Verifying the Archive

```
SELECT * FROM ProductArchive;
```

```
154 AFTER DELETE ON Products  
155 FOR EACH ROW  
156 BEGIN  
157     INSERT INTO ProductArchive (ProductID, ProductName, Category, Price, DeletedAt)  
158     VALUES (OLD.ProductID, OLD.ProductName, OLD.Category, OLD.Price, NOW());  
159 END //  
160  
161 DELIMITER ;  
162
```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	ProductID	ProductName	Category	Price	DeletedAt
▶	3	Chair	Furniture	2500.00	2026-06-28 18:31:04

ProductArchive 17 x